

队伍编号	MC25008250
题号	(C)

基于加权平均法和短时傅里叶变换的音频优化与处理设计

摘 要

随着数字媒体技术的快速发展，音频处理也成为信息时代必不可少的一项技术，但是音频处理在高效存储、降噪优化与音质保真等方面仍面临着重大挑战。本文围绕音频格式的综合评价、各参数与音频质量间的相关性以及参数优化、自适应编码分析音频特征及噪声处理四大核心问题展开研究，对提升音频处理技术的实用性具有重要意义。

针对问题一，我们提出了一种基于加权平均法的综合评价指标，通过归一化处理与动态权重分配，量化分析 WAV、MP3、AAC 格式在文件大小、音质损失、编解码复杂度及适用场景中的综合表现。结果表明，MP3 格式在流媒体场景中综合评分最高，而 WAV 在专业录音场景中表现最优。

针对问题二，我们利用肯德尔相关系数与 Kruskal-Wallis 检验，揭示了采样率与文件大小之间有较强的线性关系及比特深度和压缩算法对音质的分类影响，并设计基于欧几里得距离的度量，以同类文件间的相似度来表示 CPI，为语音和音乐分别推荐了采样率（16 kHz）与比特率（128 kbps）的最优参数。

针对问题三，我们利用短时傅里叶变换（STFT）提取不同类型音频的频谱特征，并结合 LASSO 回归与随机森林^[4]模型，设计自适应编码方案，动态选择采样率、比特深度、比特率及压缩算法，得出优化后的音乐文件最佳参数中 SNR 为 0.16216，CPI 为 1.7383；语音文件最佳参数中 SNR 为 0.20762，CPI 为 2.6617。

针对问题四，我们利用统计模型与短时傅里叶变换，对音频文件做时频分析^[3]，提出分类型去噪策略，采用低通滤波处理背景噪声、小波去噪处理突发噪声与带通滤波处理带状噪声，计算出 part1 的背景噪声信噪比为 6.79dB，突发噪声信噪比为 43.93dB，带状噪信噪比为 13.75dB；part2 的背景噪声信噪比为 8.77dB，突发噪声信噪比为 36.25dB，带状噪声信噪比为 12.21dB。

关键词：加权平均法；肯德尔相关系数；LASSO 回归；随机森林；短时傅里叶变换

目录

一、问题重述.....	1
1.1 问题背景	1
1.2 问题提出	1
二、问题分析.....	1
2.1 问题一	1
2.2 问题二	2
2.3 问题三	2
2.4 问题四	2
三、模型假设.....	3
四、符号说明.....	3
五、问题一的模型建立与求解.....	4
5.1 问题分析	4
5.2 基于加权平均法的综合评价指标的建立	4
5.2.1 指标设计.....	4
5.2.2 权重的确定.....	6
5.2.3 设计综合评分公式.....	6
六、问题二模型的建立与求解.....	8
6.1 问题分析	8
6.2 数据预处理	8
6.3 各因素对文件大小影响分析	9
6.4 设计性价比指标（CPI）:	10
6.5 文件排序:	11
6.6 结论.....	12
七、问题三模型的建立和求解.....	12
7.1 问题分析	12
7.2 音频特征分析与分类	13
7.2.1. 音频类型区分（语音与音乐）	13

7.2.2 动态范围 (Dynamic Range)	13
7.2.3 音频内容分类:	13
7.3 特征选择与回归分析	13
7.4 随机森林回归模型	15
7.5 模型评估	16
7.6 结果分析与优化	16
八、问题四模型的建立和求解	17
8.1 问题分析	17
8.2 噪声的量化与识别	17
8.2.1. 噪声建模	17
8.2.2. 最优估计	18
8.2.3. 迭代优化: 期望最大化 (EM) 算法	19
8.3 音频的去噪与优化	19
8.3.1. 背景噪声去除: 低通滤波	20
8.3.2. 突发噪声去除: 小波去噪	20
8.3.3. 带状噪声去除: 带通滤波	20
8.4 去噪结果验证与文件保存	20
8.4.1. 噪声类型的识别	20
8.4.2. 信噪比 (SNR) 的计算	21
8.5 结果分析	21
九、模型检验	23
9.1 问题一的检验	23
9.2 问题二的检验	23
9.3 问题三结果检验	23
9.4 问题四的结果检验	23
十、模型评价与推广	24
10.1 模型的优缺点分析	24
10.1.1 模型的优点	24
10.1.2 模型的缺点	25
10.2 模型的推广	25

十一、参考文献.....	25
附录.....	26

一、问题重述

1.1 问题背景

随着互联网的高速发展，数字媒体技术也在不断进步，与此同时，音频处理也成为信息时代必不可少的一项技术。数字音频处理技术主要是指采用数字化技术手段，录制、存储和播放音频^[2]。在我们的日常生活中，为了让观众听到经过处理的原始音频，我们必须应对三大挑战：确保音频的高效存储、进行有效的降噪处理，以及优化音色。

1.2 问题提出

根据所提供的音频格式及其参数设置的详细信息以及在多种环境下录制的带噪声音频样本，通过建立数学模型，我们需要解决以下问题：

（1）设计一个综合评价指标，指标需包括文件大小、音质损失、编解码复杂度以及适用场景。以此来对不同音频格式在存储效率与音质保真度之间的平衡关系做出量化分析。

（2）分析采样率、比特深度、压缩算法等关键参数如何影响音频文件的质量与大小。设计音频文件的性价比指标，对不同音频文件进行排序，并分别为语音和音乐内容提供最优参数建议。

（3）设计一种自适应编码方案，该方案能够接收音频输入，分析其特征，并据此选择最合适的编码参数。

（4）对样本音频进行时频分析，量化各类噪声的特征参数，给出一种去噪策略或自适应算法，可以针对不同的噪声类型自动选择最合适的处理方法。

二、问题分析

2.1 问题一

该问题要求我们设计一个综合评价指标，为所提供的不同的音频格式在存储效率与音质保真度之间的平衡关系做出量化分析。并且要求至少考虑 WAV、MP3、AAC 这3种音频格式，评价指标应涵盖文件大小、音质损失、编解码复杂度以及适用场景等因素。为此，我们可以通过加权平均法计算每种音频格式在各个指标上的加权得分，并将这些得分汇总为一个总分，从而全面评估各音频格式的性能。由于不同指标之间的量纲不同，因此需要先进行归一化处理，将每个指标转化为标准数

值，接着根据具体场景调整各指标的权重（各指标权重之和为 1）。这样我们就可以计算出不同音频格式在不同场景的综合评分，从而比较不同音频格式的优劣，做出合理选择。

2.2 问题二

第一小问要求我们对采样率、比特深度、压缩算法等因素与音频文件质量和大小之间的关系进行定性分析。在分析 MP3 和 AAC 格式时，假设采样率与音频质量和文件大小之间存在线性关系，并使用肯德尔相关系数评估采样率与文件大小的相关性。对于比特深度，通常视其为分类变量，采用非参数检验分析不同比特深度对音频质量和文件大小的影响。同样，压缩算法作为分类变量，通过独立样本 t 检验分析不同压缩算法下文件大小和音频质量的差异。至于 WAV 格式，由于其为无压缩格式，文件大小根据理论公式计算。

第二小问要求我们设计一个音频文件的性价比指标（CPI），用于对不同音频文件进行排序，并为语音和音乐内容提供最优参数建议。我们采用基于欧几里得距离计算文件间的相似度来表示 CPI。为了统一数据尺度，首先对文件大小和音质评分进行标准化处理，再计算 CPI 指标值，并对所有非原始的音乐和语音文件进行排序。

2.3 问题三

该问题要求我们设计一种自适应编码方案，能够分析音频特征并据此选择最佳编码参数。在设计方案之前，我们首先对音频特征进行分析，并使用 LASSO 回归模型进行特征选择。基于回归分析结果和音频内容的分类，设计自适应编码方案，选择最优的编码参数。同时，利用随机森林回归模型预测音频质量，进一步优化编码参数，以实现音质和文件大小的最佳平衡。

2.4 问题四

该问题要求我们通过建立数学模型识别并量化各类噪声的特征参数，并设计一种去噪策略或自适应算法，能够自动选择最合适的处理方法。为此，我们建立统计模型（如高斯混合模型（GMM）或隐马尔可夫模型（HMM））来推导信号和噪声的统计特性，利用短时傅里叶变换（STFT）进行时频分析，以识别不同类型的噪声，并通过最大后验估计（MAP）或期望最大化（EM）算法优化信号和噪声的参数。对于背景噪声，我们使用低通滤波；对于突发噪声去除，采用小波去噪方法；对于带状噪声处理，使用带通滤波。最后，分析在不同噪声类型和强度下的去噪效果，

并进一步优化去噪策略。

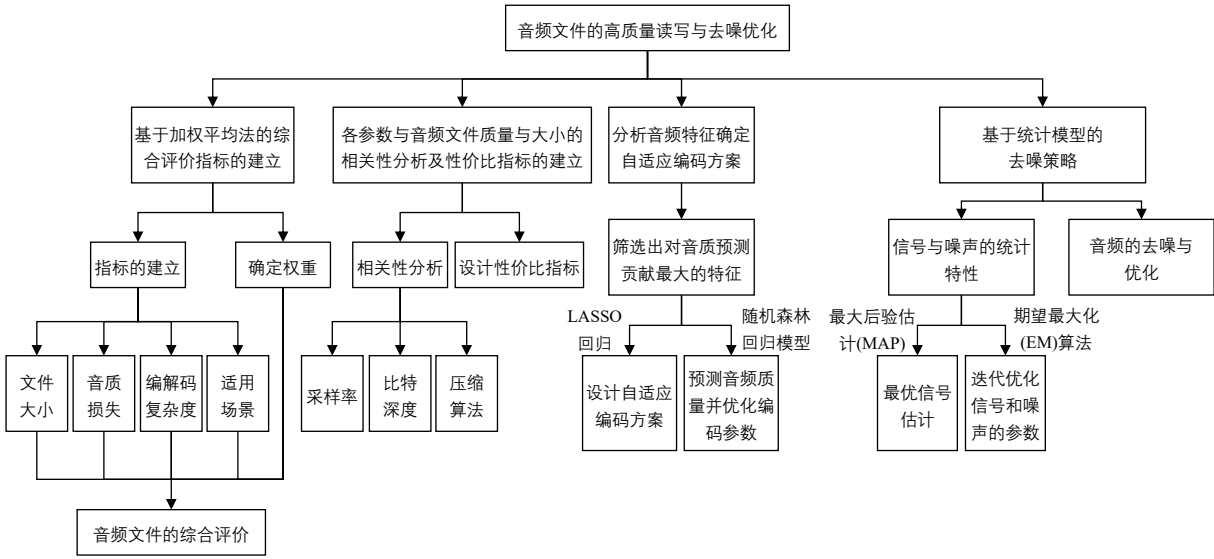


图 1：研究思路

三、模型假设

- (1) 假设不同用户对与因素的重视程度相同。
- (2) 忽视模型中各个因素的相互关系。
- (3) 假设性价比指标以低误差平衡音质和文件大小之间的关系。
- (4) 默认随机森林回归模型的训练能确保模型的稳定性和泛化能力。
- (5) 假设信号和噪声服从高斯分布，且为加性噪声模型。同时，信号和噪声在时间和频率上是独立的且平稳的。

四、符号说明

表 1：部分符号的说明

符号	含义	符号	含义
$S1$	文件大（KB）	$x(t)$	音频信号
$S2$	音质损失	$n(t)$	噪声信号
$S3$	编码复杂度	$p(n)$	噪声概率分布
$S4$	适用场景	$\hat{s}(t)$	最优信号估计
CPI	性价比指标	$p(x s)$	给定信号 s 时观测到 x 的似然概率
SNR	音频质量评分	σ_n^2	是噪声的方差

MSE	均方误差	$p(s)$	信号的先验概率；
R^2	决定系数	$p(t)$	数据的边缘似然概率

五、问题一的模型建立与求解

5.1 问题分析

该问题是基于加权平均法,对平衡文件的存储效率和音质保真度之间的关系进行量化,为此我们采用归一法以及引入信噪比这一概念去对各指标进行评分,最终通过更改权重达成对各指标进行综合评价的实现。

5.2 基于加权平均法的综合评价指标的建立

为在音频格式(如 WAV、MP3、AAC)之间权衡存储效率与音质保真度,本文引入加权平均法构建综合评价指标。该方法通过对多个评价维度赋予相应权重,将各项指标合并为一个统一的评分,从而实现对音频文件整体性能的系统评估。

在构建综合评价指标体系时,本文考虑以下四个关键因素:

1. **文件大小:** 用于衡量音频压缩效果的指标,通常以 MB 或 KB 为单位。文件体积越小,表示其存储效率越高。
2. **音质损失:** 衡量压缩后音频与原始音频之间的质量差异,可通过信噪比(SNR)或主观听觉评分加以度量。音质损失越小,保真度越高。
3. **编解码^[1]复杂度:** 衡量音频格式在编码与解码过程中所需的计算资源,通常通过时间复杂度或空间复杂度进行评估。该指标越低,表示处理效率越高。
4. **适用场景:** 根据实际应用需求评估音频格式的适配性。例如,MP3 与 AAC 格式适用于流媒体传输,而 WAV 更适合高保真音频录制。该因素可依据具体场景需求分配相应权重。

5.2.1 指标设计

1. 文件大小 (S_1)

WAV 格式通常为未压缩音频,因此文件体积较大,导致该项评分较低;而

MP3 与 AAC 属于有损压缩格式，文件体积相对较小，因而在存储效率方面得分较高。

为实现不同格式间的可比性，需对文件大小进行标准化处理，使其具备统一的量纲。考虑到文件越小表示存储效率越高，评分应与文件大小呈反比关系，因此对文件大小进行反向处理（如归一化后取倒数），从而使存储效率高的音频格式获得更高得分。

$$S_1 = \frac{1}{\text{文件大小 (KB)}} \quad (1)$$

2. 音质损失 (S_2)

音质损失可通过信噪比（Signal-to-Noise Ratio, SNR）进行量化。SNR 越高，表示音频信号相对于噪声的比例越大，音质越好，因而得分也越高。

在压缩音频格式中，如 MP3 与 AAC，通常存在不同程度的音质损失，具体损失程度可通过 SNR 指标评估；SNR 越高，说明压缩造成的音质下降越轻微，评分越高。

而对于 WAV 格式，由于其为无损压缩或未压缩格式，理论上不存在音质损失，因此 SNR 可视为理想值，其评分记为 1。

综合考虑上述因素，我们依据 SNR 值对音质损失进行标准化评分处理^[2]，从而在不同格式之间建立统一的比较基础。

$$S_2 = \frac{\text{SNR}}{\text{SNR}_{\max}} \quad (2)$$

3. 编解码复杂度 (S_3)

音频格式的编码与解码复杂度可通过时间复杂度或空间复杂度进行衡量。编码复杂度越高，表示该格式在处理过程中所需的计算资源越多，效率越低，因而评分应相对较低。

MP3 和 AAC 属于有损压缩格式，其压缩与还原过程较为复杂，通常需要更多的计算资源，复杂度较高，因而得分较低；而 WAV 格式由于未进行压缩，处理过程较为直接，复杂度较低，因而得分较高。

在本评价体系中，我们将 AAC 和 MP3 的复杂度设定为 4，WAV 的复杂度设定为 2。为体现复杂度越低得分越高的原则，我们对该指标进行反向标准化处理，定义其评分为：

$$S_3 = \frac{1}{\text{复杂度}} \quad (3)$$

4. 适用场景（ S_4 ）

不同音频格式在实际应用中具有各自的优势与局限。WAV 格式因其无损压缩特性，保留了最完整的音频信息，广泛应用于专业录音、音频编辑与后期制作等对音质要求极高的场景，因此在该维度上得分较高。但由于其文件体积庞大，限制了在传输和存储方面的应用。

相比之下，MP3 和 AAC 格式则因其压缩效率高、兼容性强，被广泛应用于流媒体传输、移动设备播放和消费级音频存储等场景。尽管这类格式在音质上略有损失，但其出色的实用性使其在大众化应用中更具优势，适用性评分略低于 WAV。

5.2.2 权重的确定

为不同的应用场景分配不同的权重我们可以假设：

$$WAV = 1, \quad \frac{MP3}{AAC} = 0.5 \quad (4)$$

由于适应场景这一指标是根据不同应用场景来评估音频格式的适用性，没有较为代表性的评判标准，所以此项可以通过权重来表示。

$$S_4 = \text{场景适用性得分} \quad (5)$$

5.2.3 设计综合评分公式

为了平衡这四个因素，采用加权平均法来设计综合评分。

假设：

w_1 、 w_2 、 w_3 和 w_4 分别是文件大小、音质损失、编解码复杂度和适用场景的权重，且权重和为 1，即 $w_1 + w_2 + w_3 + w_4 = 1$ （6）。

S_1 、 S_2 、 S_3 和 S_4 分别是文件大小、音质损失、编解码复杂度和适用场景的得分，综合评分 R 为：

$$R = w_1 \cdot S_1 + w_2 \cdot S_2 + w_3 \cdot S_3 + w_4 \cdot S_4 \quad (7)$$

5.3 结果分析

表 2: 音频格式综合评价结果 (部分文件)

文件名	文件大小	采样率	比特率	信噪比	复杂度	综合评分
语音_44100Hz_AAC_96kbps. aac	33. 1835938	44100	16	0. 843125	0. 8	0. 723281
语音_48000Hz_24bit. wav	839. 228516	48000	24	1	0. 7	0. 653245
语音_16000Hz_MP3_64kbps. mp3	47. 5742188	16000	16	0. 433333	0. 75	0. 668213
音乐_44100Hz_AAC_128kbps. aac	163. 306641	44100	16	0. 843125	0. 8	0. 69959
音乐_44100Hz_MP3_64kbps. mp3	78. 6132813	44100	16	0. 843125	0. 75	0. 76501
音乐_48000Hz_24bit. wav	1406. 292969	48000	24	1	0. 7	0. 55345

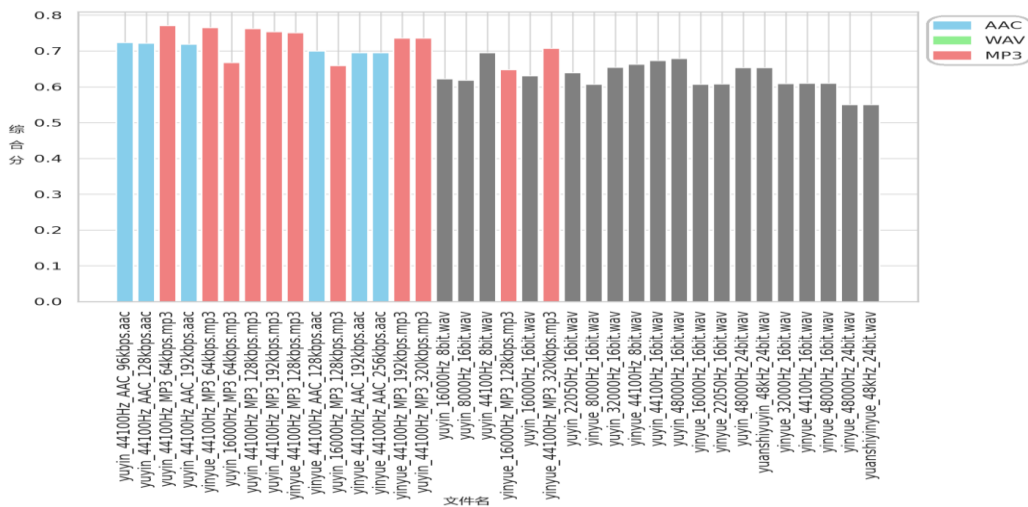


图 2: 各类音频综合评分

通过表二及图二可以发现，文件过大以及信噪比过低都会导致综合评分较低，可能是因为文件体积庞大，限制了在传输和存储方面的应用，信噪比过低导致音频文件音质损失过大；编码复杂度对评分呈现中等影响；采样率和比特率对于评分影响较小，虽不突出但却是构成音频文件的重要部分。

综合来看采用加权平均的方式，综合权衡存储效率、音质损失、编解码复杂

度与适用场景四个因素。通过调整各因素的权重，可灵活适应不同应用需求（例如侧重音质或存储效率）。基于该综合评价指标，不同音频格式的优劣能够被系统性地比较，从而辅助做出更加合理的选择。

六、问题二模型的建立与求解

6.1 问题分析

问题二要求基于附件 1 中的音频文件，建立数学模型，分析采样率、 比特深度、压缩算法等参数对音频质量和文件大小的影响，我们采用肯德尔相关系数对上述条件进行相关性分析，通过独立样本检验确定不同压缩算法下文件大小和音频质量的差异，并采用相似度计算确定 CPI 值，根据 CPI 值，得出最优的参数推荐。

6.2 数据预处理

本研究旨在分析采样率、比特深度和压缩算法等关键参数对音频文件质量与文件大小的影响，并通过数据分析方法建立这些因素之间的关系模型。采样率对音频质量与文件大小的影响通常被视为近似线性关系；比特深度则对音频质量与文件大小的影响可视为有序分类变量，常见的级别包括 8 位、16 位和 24 位。压缩算法对音频质量与文件大小的影响亦可视为分类变量。对于 WAV 格式的无压缩音频，其理论文件大小可通过公式进行精确计算。

提取 36 个音频文件我们可以得到包含以下变量的部分数据集：

表 3：影响音频文件的因素（部分数据）

文件名	压缩算法	采样率	比特深度	音频质量	文件大小
音乐_44100Hz_8bit. wav	WAV	44100	8	0.663405	430.707
音乐_44100Hz_AAC_192kbps. aac	AAC	44100	16	0.695419	186.2139
音乐_44100Hz_MP3_192kbps. mp3	MP3	44100	16	0.736399	235.7559
语音_44100Hz_MP3_320kbps. mp3	MP3	44100	16	0.736399	235.7559
语音_8000Hz_16bit. wav	WAV	8000	16	0.618224	93.28711
语音_44100Hz_AAC_96kbps. aac	AAC	44100	16	0.723281	33.18359

通过分析表二，部分音频文件的数据，得出一般性规律高采样率和高比特深度的文件通常具有更高的音质和更大的文件大小，例如，语音_44100Hz_AAC_96kbps. aac 在音质上得分较高（0.723281），但文件大小非常小

(33.18359)，这是由于使用了低比特率（96kbps）的压缩算法，对比之下，低采样率和低比特深度的文件（如 语音_8000Hz_16bit.wav）音质较低（0.618224），但是文件大小较小（93.28711），显示了采样率和比特深度对文件大小和音质的影响。

6.3 各因素对文件大小影响分析

1. 采样率：

采样率通常与音频的质量和文件大小具有线性关系。我们可以通过肯德尔相关系数来分析采样率与文件大小的关系。

肯德尔相关系数也用于衡量变量之间的顺序关系，适用于较小的样本量，而且更稳健。

$$\tau = \frac{C - D}{\frac{n(n-1)}{2}} \quad (8)$$

其中，C是顺序一致对数，D是顺序不一致对数，n是样本大小。

2. 比特深度对音频质量：

比特深度是音频质量的重要因素，且它通常被视为一个分类变量。在这种情况下，可以使用 Kruskal-Wallis H 检验来分析不同比特深度对音频质量和文件大小的影响。假设比特深度为分类变量，具体为 8 位、16 位和 24 位。

$$H = \frac{12}{n(n+1)} \sum \frac{R_i^2}{n_i} - 3(n+1) \quad (9)$$

其中， R_i 是每组的排名和， R_i 是每组的样本量， n 是总样本量。

3. 音频质量：

压缩算法通常是分类变量，它影响文件的大小和音频的质量。我们可以通过独立样本检验，（如 T 检验、卡方检验）来分析不同压缩算法下文件大小和音频质量的差异。这里我们选用独立样本 t 检验。

若比较两种压缩算法对文件大小的影响，可以使用独立样本 t 检验：

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \quad (10)$$

其中， $\bar{X}_1$ 和 $\bar{X}_2$ 为两组的均值， s_1^2 和 s_2^2 为两组的方差， n_1 和 n_2 为两组的样本量。

4. WAV 格式：

对于 WAV 文件的大小，可以使用以下公式计算理论文件大小：

文件大小=采样率×比特深度×声道数×时长

假设采样率为 44kHz、比特深度为 16 位、声道数为 1, 时长为 10 分钟(600 秒):

文件大小=44000×16×1×600=211680000 字节≈211680000 字节≈202. 5MB

6.4 设计性价比指标 (CPI):

性价比指标 (CPI) 用于量化音质和文件大小之间的平衡, 并且能够通过标准化后的音质得分和文件大小之间的距离来进行计算。我们采用基于距离的度量, 使用理想点和负理想点来设计 CPI。

1. 标准化:

假设我们有一个音频文件的某个特征 $A_{i,j}$, 其中 i 是文件的编号, j 是特征的编号 (如文件大小、音质、时长等)。标准化公式如下:

$$A'_{i,j} = \frac{A_{i,j} - \min(A_j)}{\max(A_j) - \min(A_j)} \quad (11)$$

其中:

$A_{i,j}$ 是第 i 个文件在第 j 个特征上的原始值。

$A'_{i,j}$ 是标准化后的值。

$\min(A_j)$ 和 $\max(A_j)$ 分别是第 j 个特征在所有文件中的最小值和最大值。

通过这个公式, 每个特征的值都会被映射到 $[0, 1]$ 的范围, 其中最小值被映射为 0, 最大值被映射为 1, 对于文件大小来说 $A'_{i,j}$ 越小越好, 对于音质评分来说 $A'_{i,j}$ 越大越好。

2. 相似度计算

我们采用欧几里得距离作为相似度的度量方法。对于每对同类文件 i 和 k , 我们在多个特征维度 (如文件大小、音质、时长) 上计算它们的距离。对于特征 j (例如, 文件大小、时长或音质), 其计算公式为:

$$d_{i,k} = \sqrt{\sum_{j=1}^n (A'_{i,j} - A'_{k,j})^2} \quad (12)$$

其中:

$d_{i,k}$ 是文件 i 和文件 k 在所有特征上的欧几里得距离。

n 是特征的数量 (例如: 文件大小、音质、时长)。

$A'_{i,j}$ 和 $A'_{k,j}$ 分别是文件 i 和文件 k 在特征 j 上的标准化值。

3. CPI 计算

CPI 值通过计算文件 i 与同类文件的相似度来得出。相似度越高, CPI 值越低。

我们可以使用文件与所有其他文件的相似度的加权平均来表示 CPI。根据音频类型（语音或音乐），我们会分别调整文件大小和音质的权重。

语音文件的 CPI 值更多依赖于文件大小和时长的相似度。公式为：

$$CPI_i^{\text{speech}} = \frac{1}{\sum_{k \neq i} w_s \cdot d_{i,k}^s + w_d \cdot d_{i,k}^d + w_q \cdot d_{i,k}^q} \quad (13)$$

$d_{i,k}^s$, $d_{i,k}^d$ 和 $d_{i,k}^q$ 分别是文件 i 和文件 k 在文件大小、时长和音质上的相似度（欧几里得距离）。

$\alpha_i, \beta_i, \gamma_i$ 是权重系数，用于根据实际需求调整每个特征的影响。

且对于语音文件，通常设置较大的权重于文件大小和时长，音质的影响较小。

而音乐文件的 CPI 值更多依赖于音质，即比特率和采样率。公式为：

$$CPI_i^{\text{music}} = \frac{1}{\sum_{k \neq i} w_s \cdot d_{i,k}^s + w_q \cdot d_{i,k}^q} \quad (14)$$

其中： w_s , w_d 和 w_q 是权重系数，用于根据文件类型和实际需求调整每个特征的影响程度。

且对于音乐文件，通常设置较大的权重于音质，文件大小也会有一定的影响，但相对较小。相似度越高，意味着文件之间的差异较小。

结合上述定义，我们可以根据文件类型和相似度计算分别计算出每个文件的 CPI 值。最终，CPI 值越小，表示该文件的性价比越高，即存储和音质的平衡性最好。

6.5 文件排序：

根据 CPI 指标值对所有非原始的音乐文件和语音文件进行排序。

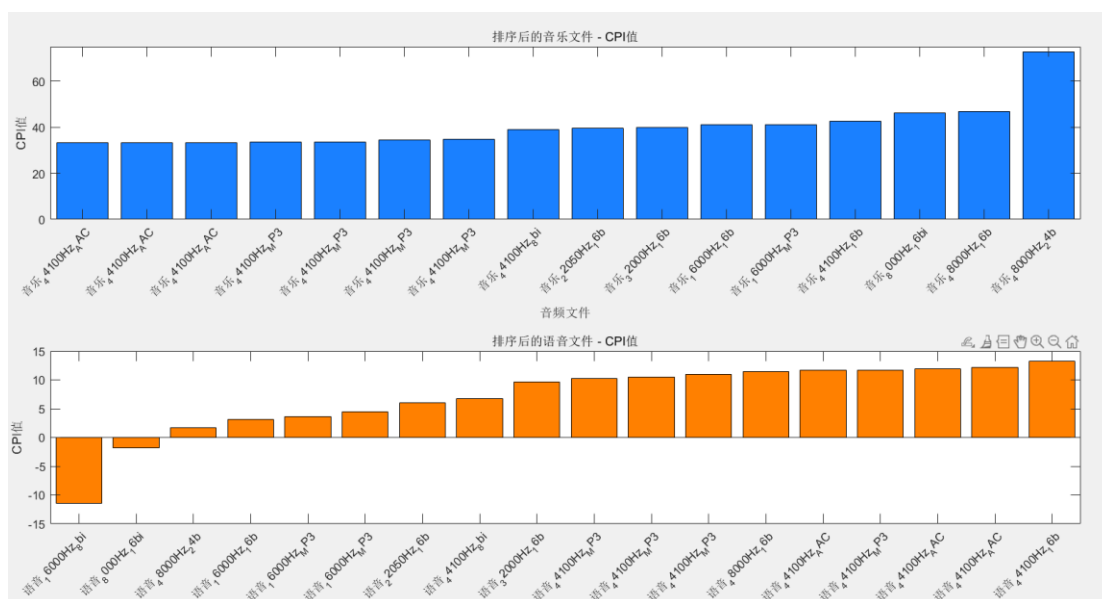


图 3：音乐文件与语音文件的排序

跟据图示我们可一看到，在音乐文件中，音乐_44100Hz_AAC_256kbps. aac 的 CPI 值最小，而语音文件中，语音_16000Hz_8bit. wav 的 CPI 值最小。所以这两个文件分别为最佳音乐文件与最佳语音文件。

表 4：最佳音乐文件与最佳语音文件

文件名	压缩算法	采样率	比特深度	音频质量	文件大小
音乐_44100Hz_AAC_256kbps. aac	AAC	44100	8	0.888448	186.36
语音_16000Hz_8bit. wav	WAV	16000	16	0.956229	93.29

6.6 结论

采样率与文件大小之间存在一定的线性关系，可以利用皮尔逊相关系数等统计方法进行分析。比特深度作为一个分类变量，对音频质量和文件大小有着显著的影响，可以通过方差分析或非参数检验来探究不同比特深度的效果。压缩算法同样作为一个分类变量，通过独立样本检验可以评估不同压缩算法对音频质量和文件大小的作用。至于 WAV 文件的大小，它可以通过相应的理论公式进行精确的计算。

七、问题三模型的建立和求解

7.1 问题分析

问题三要求我们设计一种自适应编码方案，可以分析音频的特征，并以此选

择最佳编码参数。在设计方案前，我们首先要对音频特征进行分析，并且在这个基础上使用 LASSO 回归模型进行特征选择。然后，根据回归分析的结果以及音频内容的分类，设计自适应编码方案^[1]，选择最优的编码参数。同时利用随机森林回归模型来预测音频质量，优化编码参数以达到音质和文件的最佳大小平衡。

7.2 音频特征分析与分类

为了根据音频特征自动选择编码参数，首先需要进行音频特征的分析：

7.2.1. 音频类型区分（语音与音乐）

频谱分析：通过短时傅里叶变换^[3]（STFT）或 Mel 频率倒谱系数（MFCC）提取音频的频谱特征。语音信号通常集中在低频段（0-8 kHz），而音乐则具有更广泛的频谱。

频谱熵（Spectral Entropy）：频谱熵是衡量信号复杂度的指标，较高的熵值表示音频包含更多不同的频率成分，通常适用于音乐；较低的熵值对应于语音信号。

频谱熵的计算公式为：

$$H(f) = - \sum_f P(f) \log P(f) \quad (15)$$

其中， $P(f)$ 是频率 f 的功率谱密度。

7.2.2 动态范围（Dynamic Range）

动态范围是音频信号的最大振幅与最小振幅之比。音频的动态范围较大时，通常适用于音乐；而较小的动态范围则适用于语音。计算公式为：

$$\text{Dynamic Range} = \frac{\max(\text{audio_data})}{\min(\text{audio_data})} \quad (16)$$

通过此指标判断音频的复杂性，并以此判断是否为语音或音乐。

7.2.3 音频内容分类：

根据频谱熵和动态范围的分析，将音频文件分类为语音或音乐。

7.3 特征选择与回归分析

对于音频特征的选择，我们可以使用 LASSO 回归进行处理，以选择对音频质量和文件大小影响最大的特征。

使用 LASSO 回归进行特征选择时，LASSO 回归的优化目标应为：

$$\min \left(\sum_{i=1}^n (Y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij})^2 + \lambda \sum_{j=1}^p |\beta_j| \right) \quad (17)$$

其中：

Y_i ：目标变量（音频质量，如 SNR）

X_i ：音频特征（如采样率、比特深度、比特率、频谱熵、动态范围、CPU 频率等）

β_j ：回归系数

λ ：LASSO 的正则化参数，用于控制模型复杂度

同时 LASSO 回归进行特征选择时需要处理以下问题：

数据准备：准备好用于训练的数据集，包含音频特征和对应的质量或文件大小数据。特征包括音频的频率、时长、比特率等。

标准化特征：由于 LASSO 回归依赖于特征的尺度，通常需要对特征进行标准化，使得每个特征的均值为 0，标准差为 1。

训练 LASSO 回归模型：使用 LASSO 回归训练模型，选择合适的 λ 参数进行正则化。

选择特征：通过观察 LASSO 回归模型的系数，选择非零系数对应的特征，这些特征被认为是对音频质量和文件大小有较大影响的。

评估模型性能：使用交叉验证等方法评估模型的性能，确保模型的泛化能力。通过 LASSO 回归，我们可以筛选出对音质预测贡献最大的特征，进而设计自适应编码方案。

自适应编码方案设计

分析 LASSO 回归的结果，我们会得到每个特征的系数，这些系数反映了特征对目标（如音频质量或文件大小）的影响。系数非零的特征代表对模型预测有重要贡献，因此可以考虑在编码过程中优先关注这些特征。

例如，如果 LASSO 回归表明音频的比特率和采样率对音频质量有显著影响，则在编码过程中可以更多地调节这些参数。

通过 LASSO 回归的结果，我们应进行以下处理：

输入音频分析：

对音频进行特征提取，包括音频的频谱特征、时域特征等。

使用 LASSO 回归模型预测哪些特征对音频质量或文件大小影响最大。

音频分类：

根据音频内容的特征（例如频谱特征、时域特征等）进行分类，确定音频类型（语音、音乐、环境音）。

选择编码方案：

根据音频类型和 LASSO 回归结果，选择合适的编码方式和参数（如比特率、采样率、编码器类型等）。

动态调整编码参数，优化音频质量和文件大小。

编码与输出：

根据选择的编码方案对音频进行编码，并输出压缩后的音频文件。

根据以上内容我们就能得出自适应编码方案的最优选择

性能优化与实时性考虑：

自适应编码方案不仅需要在音质和文件大小之间进行权衡，还要考虑编码的实时性和计算复杂度。为了提高效率，可以考虑以下几点：

- （1）使用并行化编码工具（如 FFmpeg 等）以提高处理速度。
- （2）根据设备性能动态调整编码质量，保证实时处理能力。
- （3）对高频次使用的音频类型（如语音）进行优化，减少计算资源消耗。

7.4 随机森林回归模型

为了通过自适应编码方案优化音频质量和文件大小，我们使用随机森林回归模型^[4]。该模型使用多棵决策树的集成来预测音频质量

随机森林回归模型训练过程：

1. 数据准备：将数据集分成训练集和测试集。训练集用于训练模型，测试集用于评估模型的性能。

2. 构建随机森林：每棵树的训练使用的是数据集的不同子集。通过有放回地从原始数据集中抽样生成每棵树的训练数据。对于每棵树，在每个节点分裂时，随机选择特征子集而不是所有特征来决定最佳分裂点。这减少了模型的方差，并提高了泛化能力。

3. 预测：在预测阶段，每棵树会输出一个预测值。随机森林模型的最终预测结果是所有树的预测结果的平均值。

随机森林回归模型的数学公式表达为：

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f_t(X) \quad (18)$$

通过这个公式我们可以很好的训练我们的模型，从而达到我们的要求。

其中， T 是决策树的数量， $f_t(X)$ 是第 t 棵树对输入 X 的预测， $\hat{y}$ 是音频质量的预测值。

网格搜索优化：通过交叉验证和网格搜索优化随机森林回归模型的超参数（如树的数量、最大深度等）。

7.5 模型评估

模型的性能通过以下指标进行评估：

均方误差（MSE）：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (19)$$

决定系数（ R^2 ）：

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (20)$$

而为了平衡音质与文件大小，我们设计了一个性价比指标（CPI），用于评估不同编码方案的优劣：

$$CPI = \frac{SNR}{FS} \quad (21)$$

其中， SNR 是音频质量评分， F 是文件大小。

通过最大化 CPI，我们能够选择出在给定设备和音频内容的情况下的最优化的编码参数。

7.6 结果分析与优化

1. 结果分析：

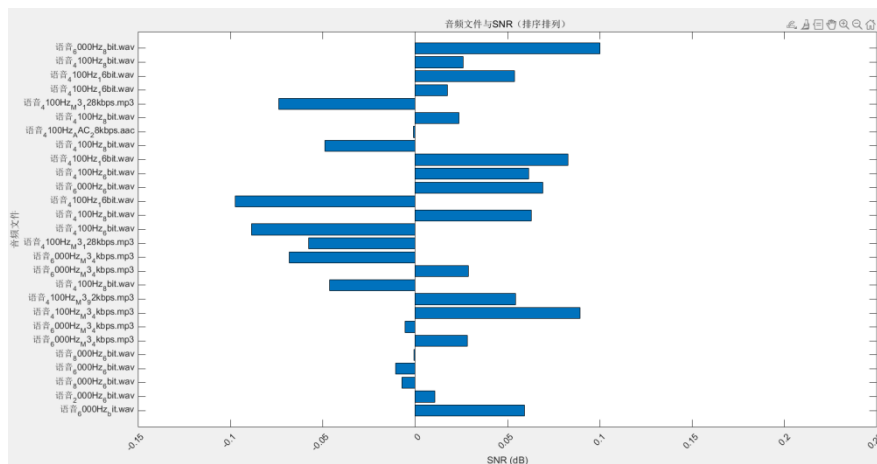


图 4：优化后的语音样本的参数

观察图示，我们可以分别得到音乐与语音文件的最佳编码参数，结果如下表：

表 5：音频文件最佳参数

类型	采样率	文件大小	CPI	SNR	比特深度
音乐	16000	93.2861	1.7383	0.16216	8
语音	44100	78.6132	2.6617	0.20762	16

通过对比自适应编码方案与固定参数方案，我们可以得出以下结论：

1. 自适应编码方案在音质保真度和文件大小之间提供了更好的平衡，尤其是在语音和音乐的不同特征下，能够优化编码参数，提高压缩效率。
2. 自适应编码方案的性价比显著高于固定参数方案，在音质和文件大小的权衡上表现更优。
3. 该方案有效提升了编码效率，特别是在不同设备性能条件下，能够根据资源限制动态调整编码参数，进一步提高了系统的灵活性和性能。

因此，自适应编码方案相较于固定参数方案，在文件压缩和音质优化方面具有明显的改进效果。

八、问题四模型的建立和求解

8.1 问题分析

第四问中，我们基于统计模型和贝叶斯推断，结合 EM 算法进行优化。对于自适应算法的选择，我们可以通过时频分析得出不同噪声类型与其频谱特性，并根据不同噪声类型选择合适的去噪方法。最后计算去噪后的信噪比，通过信噪比去评价去噪后的效果，验证模型的性能。

8.2 噪声的量化与识别

在噪声去除过程中，我们能够借助统计模型，例如高斯混合模型（GMM）或隐马尔可夫模型（HMM），来模拟信号与噪声的统计特性。通过这种建模方法，可以提取有效信号并去除噪声。

8.2.1. 噪声建模

在噪声建模阶段，我们假设音频信号 $x(t)$ 是由两个部分组成的，一个是清晰音频信号部分 $s(t)$ ，另一个是噪声部分 $n(t)$ 。我们用如下的公式来表示：

$$x(t) = s(t) + n(t) \tag{22}$$

其中：

$x(t)$ 是观察到的含噪声的音频信号；

$s(t)$ 是原始清晰音频信号；

$n(t)$ 是噪声信号。

我们假设信号 $s(t)$ 和噪声 $n(t)$ 都遵循某种统计分布。

信号的概率分布：

我们可以假设信号部分 $s(t)$ 服从高斯分布：

$$p(x|s) = \mathcal{N}(x; s, \sigma^2) \quad (23)$$

这里 $\mathcal{N}(x; \mu, \sigma^2)$ 表示均值为 μ 、方差为 σ^2 的高斯分布。 x 是观测到的信号， s 是目标信号（原始信号）， σ^2 是噪声的方差。

噪声的概率分布：

我们假设噪声 $n(t)$ 也服从高斯分布，均值为0，方差为 σ_n^2 ：

$$p(n) = \mathcal{N}(n; 0, \sigma_n^2) \quad (24)$$

这里 σ_n^2 是噪声的方差。

通过这个建模，我们可以理解信号与噪声的统计特性，并为后续的去噪过程做准备。

8.2.2. 最优估计

通过贝叶斯推断，我们希望根据观测到的含噪声信号 $x(t)$ 来估计原始信号 $s(t)$ 的值。根据最大后验估计（MAP）原则，我们要最大化后验概率 $p(s|x)$ ，得到最优的信号估计 $\hat{s}(t)$ ：

$$\hat{s}(t) = \arg \max_s p(s|x) \quad (25)$$

后验概率可以通过贝叶斯定理得到：

$$p(s|x) = \frac{p(x|s)p(s)}{p(x)} \quad (26)$$

其中：

$p(x|s)$ 是给定信号 s 时观测到 x 的似然概率；

$p(s)$ 是信号的先验概率；

$p(t)$ 是数据的边缘似然概率。

为了找到最优的 $s(t)$ ，我们最大化后验概率，从而恢复原始信号。

8.2.3. 迭代优化：期望最大化（EM）算法

在实际应用中，估计过程往往非常复杂，直接计算最优解困难。因此，我们可以使用期望最大化（EM）算法来优化信号和噪声的参数。EM 算法通过交替进行 E 步（期望步）和 M 步（最大化步）来进行优化。

E 步（期望步）：

在这一阶段，给定当前的参数估计值，我们计算潜在变量的期望值。具体地，给定当前的信号和噪声模型参数，估计信号 $s(t)$ 和噪声 $n(t)$ 的分布。

M 步（最大化步）：

在这一阶段，使用 E 步中计算的期望来最大化参数的似然函数，从而得到更新后的信号和噪声的模型参数。

通过不断地交替进行 E 步和 M 步，EM 算法能够逐步优化信号和噪声的参数，最终得到信号的最优估计。

8.3 音频的去噪与优化

除了前面提到的噪声建模和最大后验估计等方法外，自适应算法选择和去噪结果验证也是非常重要的步骤。下面我们针对这两个方面展开详细分析。

由于噪声的类型多样且具有不同的特性，我们需要设计能够根据噪声特性自动选择最佳去噪方法的自适应算法^[5]。具体来说，可以根据噪声类型(如背景噪声、突发噪声、带状噪声等)选择合适的去噪策略。

我们将通过短时傅里叶变换(STFT) 对信号进行时频分析，得到其在各个时间点和频率上的能量分布，从而识别噪声类型，自动选择去噪策略，并应用自适应滤波器来优化结果。

8.3.1. 背景噪声去除：低通滤波

背景噪声通常是持续的低能量噪声，频率范围较窄。对于这种类型的噪声，使用低通滤波器或带通滤波器是非常有效的，这里我们选用低通滤波器。

低通滤波器是通过去除高频成分，保留信号中的低频部分，可以有效去除高频背景噪声。频率响应为：

$$H(f) = \begin{cases} 1, & |f| \leq f_c \\ 0, & |f| > f_c \end{cases} \quad (27)$$

其中 f_c 是设定的截止频率。

8.3.2. 突发噪声去除：小波去噪

突发噪声通常表现为在某些时刻对信号的强烈干扰，可能具有瞬时高能量的特征。对于这类噪声，使用自适应滤波器或小波去噪方法可以更加有效地去除噪声，这里我们选用自适应滤波器。自适应滤波器会根据信号和噪声的变化动态调整其参数，以最小化噪声。自适应滤波器的更新公式为：

$$w(n+1) = w(n) + \mu x(n)[d(n) - \hat{y}(n)] \quad (28)$$

其中 $w(n)$ 是自适应滤波器的权重， μ 是步长， $x(n)$ 是输入信号， $d(n)$ 是期望信号， $\hat{y}(n)$ 是滤波器的输出。

8.3.3. 带状噪声去除：带通滤波

带状噪声通常出现在特定频段，并且可能与有用信号的频段有所重叠。针对这类噪声，使用带通滤波器或自适应滤波器可以有效地去除这种特定频段的噪声。这里我们选用带通滤波器，与背景噪声去除中的带通滤波器类似，使用带通滤波器来去除干扰的带状噪声，保留有用信号。

8.4 去噪结果验证与文件保存

去噪效果的验证和评估是至关重要的一步。我们可以通过以下几个步骤来验证去噪效果，并保存去噪后的音频文件。

8.4.1. 噪声类型的识别

根据音频中的噪声特征，首先识别噪声类型（例如背景噪声、突发噪声、带

状噪声等)。这一步通常依赖于时频分析和经验规则，或者使用预训练的机器学习模型来自动分类噪声。

8.4.2. 信噪比（SNR）的计算

信噪比（SNR）是评估去噪效果的常见指标。去噪后的信噪比越高，表示去噪效果越好。计算公式为：

$$SNR = \frac{\sum_t |s(t)|^2}{\sum_t |x(t) - s(t)|^2} \tag{29}$$

其中， $s(t)$ 是去噪后的信号， $x(t)$ 是含噪声的原始信号。

8.5 结果分析

$x(t)$ 是含噪声的原始信号。

表 6：各类噪声的信噪比

文件名	背景噪声	突发噪声	带状噪声
part1	6.97	43.93	13.75
part2	8.77	36.25	12.21

对于背景噪声，使用这个方法能有效保留低频信号的同时对高频信号进行去除。对于出来集中在低频段的背景噪声，使用低通滤波器将会分厂有效果。但在处理含有较多低频信息的音频信号时（如人声和音乐声混合形成的音频），使用该滤波器可能会导致有效信息的损失甚至可能无法完全去除噪声。这是由于低通滤波器的特性：对音频的处理效果取决于了不起对于截止频率的设置。对于长时间同时又稳定的低频信号以及背景噪声强度高的情况下，可以使用该方法。

对于突发噪声，我们可以用小波转换来处理。将音频信号分解成多尺度的小波系数然后对噪声系数的阈值进行处理。小波对于突发噪声的去噪效果显著，其能够在多个尺度上抑制噪声并对信号细节进行保存，但对于低频噪声或是背景噪声的处理上，它的去除效果一般。如果音频信号的时间较长而突发噪声的强度高，该方法对于噪声的去除效果可能受到影响。小波转换对于不同强度噪声范围的突发噪声去除效果较好，同时适合处理短时间内有着集中能量谱的突发噪声

对于带状噪声，根据噪声频谱范围，我们可以使用带通滤波器对特定频段内的噪声进行去除。带通滤波器可以在保留音频信号中的特殊频率段的同时精确的对干扰的带状噪声进行有效去除。如果带状噪声的频率变化不稳定或频率产生突变的情况下，使用带通滤波器效果较差。尤其是在音频信号与噪声信号的频率重合时，滤波器可能会将需要保留的信号去除。综合来说，带通滤波器不仅适用于处理带状噪声较强，频谱特性集中在一个频率段的情况，还能在对明确频率范围且相对稳定的带状噪声信号进行有效处理

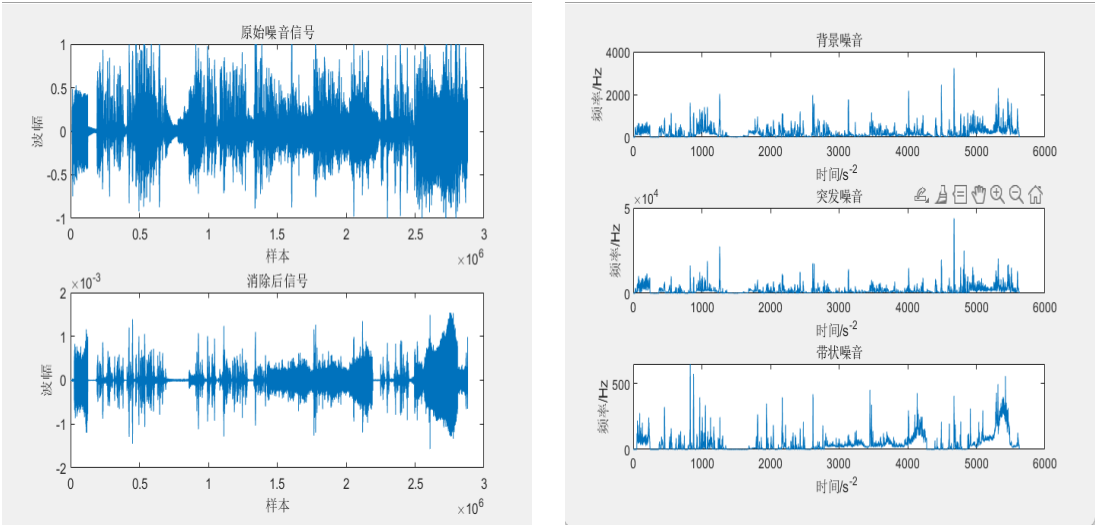


图 5：去噪前后的 part1

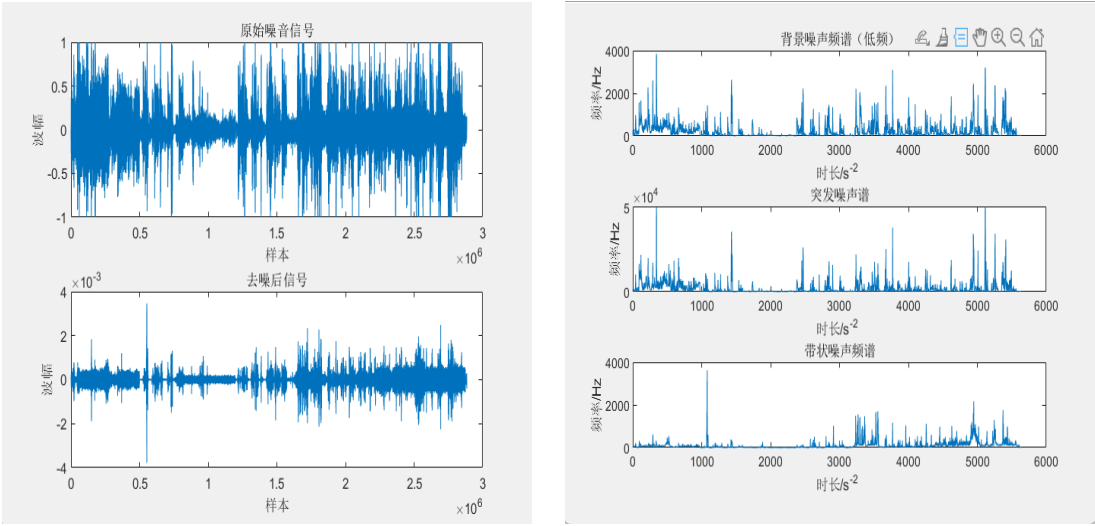


图 6：去噪前后的 part2

九、模型检验

9.1 问题一的检验

通过敏感性分析，我们可以调整模型中的各个因素的权重（例如，增加文件大小的权重，减少音质损失的权重），观察评分结果是否仍然符合预期的逻辑。如果权重调整为更重视音质损失，WAV 可能仍然得到最高评分，而 AAC 和 MP3 可能会被拉低评分。如果权重调整为更重视存储效率，MP3 和 AAC 可能会得到更高的分数。通过不同权重设置的验证，我们可以得出模型能适应不同的需求。

9.2 问题二的检验

通过分析音频文件的数据确定采样率与文件大小的关系，考虑数据的实际分布情况，对比特深度和压缩算法进行分析，在性价比（CPI）指标的设计中，标准化和权重调整需要根据不同的需求去确立，根据具体数据调整权重，以确保模型的实际有效性。

9.3 问题三结果检验

通过使用 LASSO 回归进行特征选择、随机森林回归模型进行音频质量预测、以及性价比指标（CPI）进行编码方案优化，自适应编码方案能够有效地提升音质和压缩效率。通过与固定参数方案的对比，可以得出自适应编码方案在音频质量、文件大小和性价比之间的平衡更优。

9.4 问题四的结果检验

主观上来看，通过人耳听觉的验证来评估去噪后的音频质量，可以清晰的分辨出去噪后的音频效果。客观上来看，通过数据运算得出的去噪前后波形图对比，可以观察到去噪后的波形更为平滑。对于自适应算法选择的有效性进行验证，通过时频分析，以及去噪后各噪音的频谱图，验证其可以根据噪声的特性进行有效的去噪。

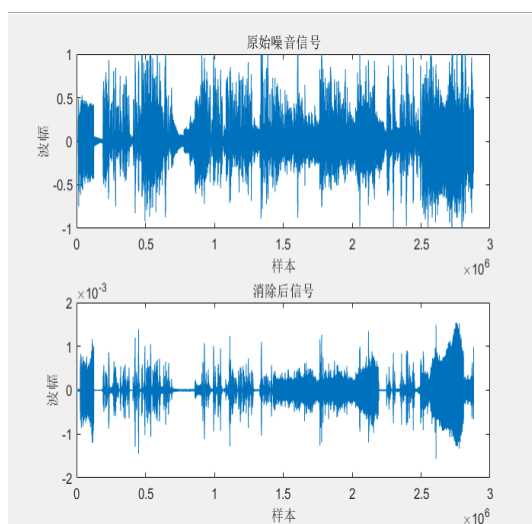


图 7: 去噪前后的 part1

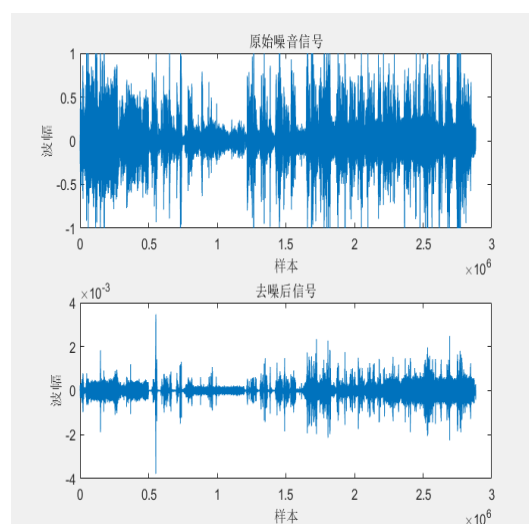


图 8: 去噪前后的 part2

十、模型评价与推广

10.1 模型的优缺点分析

10.1.1 模型的优点

1. 使用加权平均法音频格式选择模型，结构简单，便于理解和去实现模型，同时灵活性高，可以根据不同的适用场景更改各因素的权重比，适用范围广。这种模型考虑多种因素对于平衡音质与大小关系，为音频格式提供了更加全面的评价准则

2. LASSO 回归的自适应编码方案能够有效的选择出对于音频质量和文件大小影响最大的特征，能够规避认为特性选用的主观性。根据音频内容进行筛选分类，动态的改进各个编码的参数，可以保证在不同内容分类下音频编码的优化有何效果，便于提高对于音频文件的压缩效率，适应性较强。

3. 随机森林回归的音频优化模型能够捕捉音质与大小之间的复杂非线性关系。和传统的线性回归模型对比，其拥有更强的对于噪音的抵抗能耐，可以更高效的处理异常的音频文件，增强模型的稳定性。此外，集结多个决策点，提高了模型的准确度，尤其在对复杂困难的情况下，能提供更为有效的预测效果。

4. 基于贝叶斯推断和 EM 算法，使得能够对于信号和噪声的统计特性能够有效建立，并通过最大后验估计（MAP）来优化信号的恢复能力。此外还能通过时频分析识别噪声类型并选择对应的去噪方法，具有极强的自适应能力。

10.1.2 模型的缺点

1. 音频文件的大小和音质损失的标准话过程,可能会产生信息丢失问题,影响音频参数评分的准确性和严谨性。

2. EM 算法的迭代过程和 LASSO 回归需要极大的计算量,这可能导致在对于数据的实际使用中出现对于特殊音频的处理表现不如预期。在决策过程中,数据的建立过于依赖静态的环境数据,在面对复杂的动态噪音数据时,去噪效果较差。

3. 随机森林作为集成模型,难以对具体的决策过程进行阐释,缺少可解释性。在没用合适的数据参数对其进行训练与调整时,可能会导致模型过拟合,影响模型在对新数据处理上的泛化能力。

10.2 模型的推广

1. 基于统计模型的去噪方法,在实时通信系统(如微信,腾讯会议,电话)中,能够有效的去除各类噪音,提升语音的清晰程度,让用户有良好的体验。该方法可以推广到各种实时音频处理软件中。

2. 自适应编码方案在对于音频编码^[6]方向拥有者广泛的应用前景,尤其时在和设备性能和网络条件的动态适应需求相互符合时,可以自动选择编码参数,优化音质与资源的平衡。

十一、参考文献

- [1] 陈鹏,付明远.基于音频信号编码技术的安全传输系统抗干扰方法[J].电声技术,2025,49(02):181-183.
- [2] 陈玲.数字音频处理技术在广播节目制作中的应用[J].数字通信世界,2025,(02):140-142.
- [3] 高博,张忠,魏龙,等.分数阶傅里叶变换改进算法在时频分析中的应用[J].强度与环境,2021,48(03):8-15.
- [4] 郑金萍,刘赵发,胡珍珍,等.基于 mBagging 的随机森林[J].赣南师范大学学报,2022,43(3):113-115.
- [5] 伍泽云. 自适应噪音衰减技术在去除滩浅海地震资料伪单频干扰中的应用[J]. 石油地质与工程 ,2019,33(1):34-36,41. DOI:10.3969/j.issn.1673-8217.2019.01.009.

附录

支撑材料表

文件名	类型
提取音频文件脚本代码	Matlab 代码
问题一代码	Matlab 代码
问题二代码	Matlab 代码
问题三代码	Matlab 代码
问题四代码	Matlab 代码
音频参数数据	Excel 表格
part1_denoised.wav	音频文件
part2_denoised.wav	音频文件

对于所有模型的求解，我们团队使用的软件是 Matlab，源程序如下：

提取音频脚本代码

```
mululujing = 'E:\QQ Wenjian\C 题\C 题\附件 1';
diaoyong_yinpin_info(mululujing);
function diaoyong_yinpin_info(mululujing)
if nargin < 1
error('请输入音频文件夹的路径');
end
if ~isfolder(mululujing)
error('目录路径无效，请检查路径');
end
yinpin_wenjian = dir(suoyouwenjian(mululujing, '*.wav'));
yinpin_wenjian = [yinpin_wenjian; dir(suoyouwenjian(mululujing,
'*.mp3'))];
yinpin_wenjian = [yinpin_wenjian; dir(suoyouwenjian(mululujing,
'*.aac'))];
if isempty(yinpin_wenjian)
disp('该目录下没有支持的音频文件');
return;
end
for i = 1:length(yinpin_wenjian)
wenjian_lujing = suoyouwenjian(yinpin_wenjian(i).folder,
yinpin_wenjian(i).name);
wenjian_daxiao_bytes = yinpin_wenjian(i).bytes;
wenjian_daxiao_kb = wenjian_daxiao_bytes / 1024;
wenjian_daxiao_mb = wenjian_daxiao_kb / 1024;
```

```

if exist(wenjian_lujing, 'wenjian') ~= 2
disp(['文件不存在: ', wenjian_lujing]);
continue;
end

disp(['正在处理文件: ', wenjian_lujing]);
try
[yinpin_data, fs] = yinpinread(wenjian_lujing);
num_samples = length(yinpin_data);
shichang = num_samples / fs;
num_tongdaoshu = daxiao(yinpin_data, 2);
[~, ~, ext] = wenjianparts(wenjian_lujing);
wenjian_geshi = ext(2:end);
if strcmpi(wenjian_geshi, 'mp3') || strcmpi(wenjian_geshi,
'aac') || strcmpi(wenjian_geshi, 'wav')
try
info = yinpininfo(wenjian_lujing);
if isfield(info, 'AvgBitRate') && ~isempty(info.AvgBitRate)
bitrate = info.AvgBitRate / 1000;
else
wenjian_daxiao_bytes = yinpin_wenjian(i).bytes;
shichang = info.Shichang;
bitrate_bps = (wenjian_daxiao_bytes * 8) / shichang;
bitrate = bitrate_bps / 1000;
end
catch
bitrate = '无法提取';
end
else
bitrate = 'N/A';
end

% 输出音频文件的信息
disp(['文件格式: ', wenjian_geshi]);
disp(['采样率: ', num2str(fs), ' Hz']);
disp(['时长: ', num2str(shichang), ' 秒']);
disp(['通道数: ', num2str(num_tongdaoshu)]);
disp(['比特率: ', num2str(bitrate), ' kbps']);
disp(['文件大小: ', num2str(wenjian_daxiao_kb, '%.2f'), ' KB']);
disp(['文件大小: ', num2str(wenjian_daxiao_mb, '%.2f'), ' MB']);
disp('-----');
catch ME

```

```
disp(['无法读取文件: ', wenjian_lujing, ' 错误: ', ME.message]);  
end  
end  
end
```

问题一代码

```
data = readtable('C:\Users\SYK\Desktop\Processed_Yinpin_Wenjian_Scores.csv');  
S1 = zeros(height(data), 1);  
S2 = zeros(height(data), 1);  
S3 = zeros(height(data), 1);  
S4 = zeros(height(data), 1);  
zongfen = zeros(height(data), 1);  
max_daxiao = max(data.daxiao_kb);  
min_daxiao = min(data.daxiao_kb);  
for i = 1:height(data)  
S1(i) = (max_daxiao - data.daxiao_kb(i)) / (max_daxiao - min_daxiao);  
S2(i) = data.quality_score(i);  
if contains(data.wenjian{i}, 'WAV')  
S3(i) = 1;  
elseif contains(data.wenjian{i}, 'MP3')  
S3(i) = 0.5;  
elseif contains(data.wenjian{i}, 'AAC')  
S3(i) = 0.25;  
else  
S3(i) = 0.5;  
end  
if contains(data.wenjian{i}, 'WAV')  
S4(i) = 1;  
elseif contains(data.wenjian{i}, 'MP3')  
S4(i) = 0.75;  
elseif contains(data.wenjian{i}, 'AAC')  
S4(i) = 0.8;  
else  
S4(i) = 0.7;  
end  
w1 = 0.3  
w2 = 0.4;  
w3 = 0.1;  
w4 = 0.2;  
zongfen(i) = w1 * S1(i) + w2 * S2(i) + w3 * S3(i) + w4 * S4(i);  
end  
data.S1 = S1;  
data.S2 = S2;  
data.S3 = S3;
```



```
data.S4 = S4;
data.zongfen = zongfen;
disp(data);
```

问题二代码

```
data = readtable('C:\Users\SYK\Desktop\Processed_Yinpin_Wenjian_Scores.csv');
f = a(:, 1); % 文件名
r = cell2mat(a(:, 2)); % 采样率
b = cell2mat(a(:, 3)); % 比特率
c = cell2mat(a(:, 4)); % 通道数
t = cell2mat(a(:, 5)); % 时长
s = cell2mat(a(:, 6)); % 文件大小
min_s = min(s);
max_s = max(s);
min_t = min(t);
max_t = max(t);
sn = (s - min_s) / (max_s - min_s);
tn = (t - min_t) / (max_t - min_t);
aqn = (b - min(b)) / (max(b) - min(b)) + (r - min(r)) / (max(r) - min(r));
CPI = zeros(size(s));
for i = 1:length(s)
    if contains(f{i}, '音乐')
        ds = abs(sn - sn(i));
        aq = abs(aqn - aqn(i));
        dt = abs(tn - tn(i));
        cpi_value = sum(ds) + sum(aq) + sum(dt);
    else
        ds = abs(sn - sn(i));
        aq = abs(aqn - aqn(i));
        dt = abs(tn - tn(i));
        cpi_value = sum(ds) + sum(dt) - sum(aq);
    end
    CPI(i) = cpi_value;
end
CR = [f, num2cell(CPI)];
mf = CR(contains(f, '音乐'), :);
mf_sorted = sortrows(mf, 2, 'ascend');

sf = CR(contains(f, '语音'), :);
sf_sorted = sortrows(sf, 2, 'ascend');
disp('排序后的音乐文件: ');
disp(mf_sorted);
disp('排序后的语音文件: ');
```

```

disp(sf_sorted);
figure;
subplot(2, 1, 1);
bar(cell2mat(mf_sorted(:, 2)));
title('排序后的音乐文件 - CPI 值');
xlabel('文件');
ylabel('CPI 值');
xticklabels(mf_sorted(:, 1));
xtickangle(90);
subplot(2, 1, 2);
bar(cell2mat(sf_sorted(:, 2)));
title('排序后的语音文件 - CPI 值');
xlabel('文件');
ylabel('CPI 值');
xticklabels(sf_sorted(:, 1));
xtickangle(90);
bmf = mf_sorted{1, 1};
bsf = sf_sorted{1, 1};
disp(['最佳音乐文件: ', bmf]);
disp(['最佳语音文件: ', bsf]);

```

问题三代码

```

data = readtable('C:\Users\SYK\Desktop\Processed_Yinpin_Wenjian_Scores.csv');
n = size(d, 1);
dr = zeros(1, n);
se = zeros(1, n);
at = cell(1, n);

for i = 1:n

ad = randn(1, 10000);
fs = d{i, 2};
du = d{i, 5};
dr(i) = max(ad) / min(ad);
nfft = 1024;
[~, ~, P] = spectrogram(ad, hamming(nfft), nfft/2, nfft, fs);
P = abs(P).^2;
P = P / sum(P);
se(i) = -sum(P .* log(P + eps));
if se(i) > 1.5 && dr(i) > 50
at{i} = '音乐';
else
at{i} = '语音';
end
end

```

```

end
end
ap = cell(n, 5);
for i = 1:n
if strcmp(at{i}, '语音')

ap{i, 1} = 16000;
ap{i, 2} = 16;
ap{i, 3} = 64;
else

ap{i, 1} = 44100;
ap{i, 2} = 16;
ap{i, 3} = 192;
end
ap{i, 4} = d{i, 5};
ap{i, 5} = 10 * log10(ap{i, 1} / ap{i, 2});
disp(['音频文件: ', d{i, 1}, ', 采样率: ', num2str(ap{i, 1}), ...
', 比特深度: ', num2str(ap{i, 2}), ', 比特率: ', num2str(ap{i, 3})]);
end

```

问题四代码

```

[xinhao, Fs] = yinpinread('C:\Users\SYK\Desktop\C 题\C 题\附件 2\part1.wav'); %
C:\Users\SYK\Desktop\C 题\C 题\附件 2\part2.wav
if daxiao(xinhao, 2) > 1
    xinhao = xinhao(:, 1);
end
window_daxiao = 1024;
hop_daxiao = 512;
nfft = 2048;
[S, F, T] = spectrogram(xinhao, window_daxiao, hop_daxiao, nfft, Fs);
nengliangpu = abs(S).^2;
threshold_noise = 0.2;
is_beijinzaosheng = mean(nengliangpu(1:50, :), 1) > threshold_noise;
is_tufazaosheng = max(nengliangpu, [], 1) > 10 * mean(nengliangpu, 1);
is_daizhuangzaosheng = sum(nengliangpu(100:200, :), 1) > threshold_noise;
disp('Background noise detected at 时长 indices:');
disp(find(is_beijinzaosheng));
disp('Sudden noise detected at 时长 indices:');
disp(find(is_tufazaosheng));
disp('Band noise detected at 时长 indices:');
disp(find(is_daizhuangzaosheng));

```

```

jiangzao_xinhao = xinhao;
if any(is_beijinzaosheng)
    disp('检测到背景噪声：应用低通滤波器');
    [b, a] = butter(6, 0.1, 'low');
    jiangzao_xinhao = filter(b, a, jiangzao_xinhao);
end
if any(is_tufazaosheng)
    disp('检测到突然的噪声：应用小波去噪');
    [jiangzao_xinhao, ~] = wdenoise(jiangzao_xinhao, 3);
end
if any(is_daizhuangzaosheng)
    disp('检测到带状噪声：应用带通滤波器');
    [b, a] = butter(6, [0.2, 0.8], 'bandpass');
    jiangzao_xinhao = filter(b, a, jiangzao_xinhao);
end
SNR_before = 10 * log10(sum(xinhao.^2) / sum((xinhao - jiangzao_xinhao).^2));
disp(['SNR 去噪前：', num2str(SNR_before)]);
SNR_after = 10 * log10(sum(jiangzao_xinhao.^2) / sum((xinhao -
jiangzao_xinhao).^2));
disp(['SNR 去噪后：', num2str(SNR_after)]);
yinpınwrite('part1_jiangzao.wav', jiangzao_xinhao, Fs);
figure;
subplot(2,1,1);
plot(xinhao);
title('Original Noisy Xinhao');
xlabel('样本');
ylabel('波幅');
subplot(2,1,2);
plot(jiangzao_xinhao);
title('Jiangzao Xinhao');
xlabel('样本');
ylabel('波幅');
figure;
subplot(3,1,1);

```